

LEVEL 21 — Advanced Indexing in NumPy

Line-by-line topics with easy examples and outputs

1. Fancy Indexing

Fancy indexing lets you select multiple positions at once using an array/list of indexes. Here indexes 0, 2, and 4 select 10, 30, and 50.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

result = arr[[0, 2, 4]]

print(result)
```

Output:
[10 30 50]

2. Boolean Indexing

Boolean indexing selects values where a condition is True. Only 55 and 70 are greater than 40.

```
import numpy as np

arr = np.array([10, 25, 40, 55, 70])

result = arr[arr > 40]

print(result)
```

Output:
[55 70]

3. Integer Array Indexing

Integer array indexing uses integer positions stored in an array. Positions 1, 3, and 4 return 20, 40, and 50.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

indexes = np.array([1, 3, 4])

result = arr[indexes]

print(result)
```

Output:
[20 40 50]

4. Advanced Slicing

The slice starts at index 2, stops before index 9, and takes every 2nd element.

```
import numpy as np

arr = np.arange(1, 11)
```

```
result = arr[2:9:2]
```

```
print(result)
```

Output:

```
[3 5 7 9]
```

5. Combining Indexing

First `arr[1:4]` gives `[20 30 40]`. Then indexes 0 and 2 select 20 and 40 from that temporary result.

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40, 50])
```

```
result = arr[1:4][[0, 2]]
```

```
print(result)
```

Output:

```
[20 40]
```

6. Indexing with Multiple Dimensions

NumPy pairs the indexes element-by-element: (row 0, column 1) gives 20, and (row 2, column 2) gives 90.

```
import numpy as np
```

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
```

```
result = arr[[0, 2], [1, 2]]
```

```
print(result)
```

Output:

```
[20 90]
```

7. `np.ix_()`

`np.ix_()` creates an open mesh of row and column indexes, so every selected row is combined with every selected column.

```
import numpy as np
```

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
```

```
rows = [0, 2]
```

```
cols = [1, 2]
```

```
result = arr[np.ix_(rows, cols)]
```

```
print(result)
```

```
Output:
[[20 30]
 [80 90]]
```

8. np.take()

np.take() returns elements from specified indexes. It is useful when indexes are stored separately.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

result = np.take(arr, [0, 2, 4])

print(result)
```

```
Output:
[10 30 50]
```

9. np.put()

np.put() changes values at selected indexes. Index 1 becomes 200 and index 3 becomes 400.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

np.put(arr, [1, 3], [200, 400])

print(arr)
```

```
Output:
[ 10 200  30 400  50]
```

10. np.take_along_axis()

np.take_along_axis() takes values using indexes along a chosen axis. With axis=1, indexes select columns separately for each row.

```
import numpy as np

import numpy as np

arr = np.array([
    [10, 30, 20],
    [60, 40, 50]
])

indexes = np.array([
    [2, 0],
    [1, 2]
])

result = np.take_along_axis(arr, indexes, axis=1)

print(result)
```

```
Output:
[[20 10]
 [40 50]]
```

11. Quick Comparison

These techniques are the main tools for advanced NumPy indexing.

Fancy indexing	-> <code>arr[[0, 2, 4]]</code>
Boolean indexing	-> <code>arr[arr > 40]</code>
Integer indexing	-> <code>arr[np.array([1, 3])]</code>
Advanced slicing	-> <code>arr[2:9:2]</code>
<code>np.ix_()</code>	-> select every combination of rows/columns
<code>np.take()</code>	-> retrieve values by indexes
<code>np.put()</code>	-> replace values at indexes
<code>np.take_along_axis()</code>	-> retrieve values along a selected axis